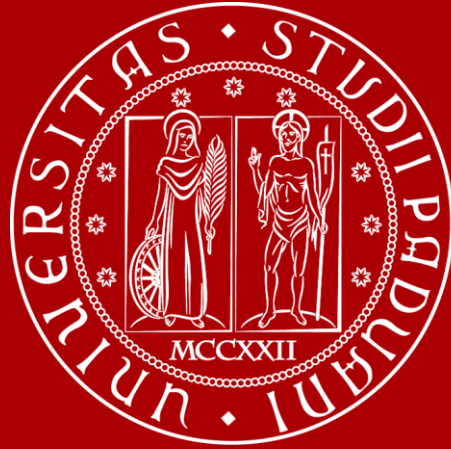




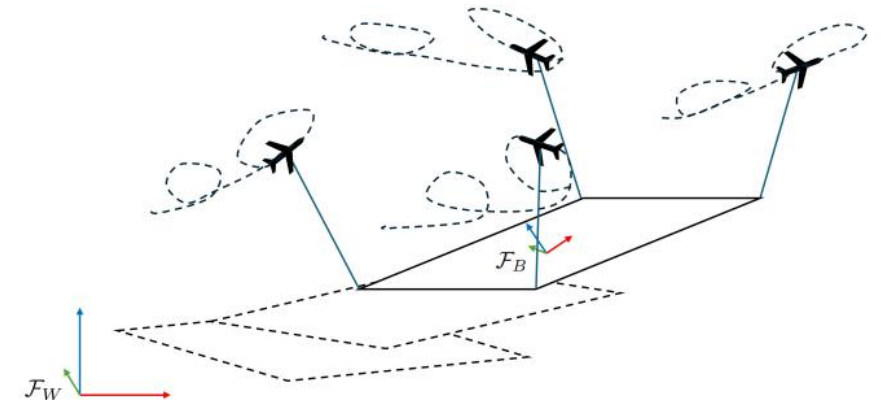
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

RL Cooperative Transportation using fixed-wings drones

RL Project
Part of thesis

Residual MARL: Problem Definition

- Four non-stopping carriers cooperatively carrying a load
- Centralized controller already exists
 - It calculates $f(t) = f_g(t) + f_\lambda(t) \in \mathbb{R}^{12}$
 - $f_g(t)$ forces applied on load
 - $f_\lambda(t)$ internal forces cancelled between the carriers (hence, they need to be coordinated). Their purpose is to keep the carriers moving to prevent stall
 - Each carrier calculates its reference position and velocity from the assigned force slice $f_i(t)$ using kinematic equations





Residual MARL: Problem Definition

- Our goal is to make this system decentralized
- The centralized controller uses global information on all the carriers
- Global carrier information is reconstructed using load state and the total applied force $f(t)$
- We can deploy the same controller on each carrier separately, each carrier can calculate the full force $f(t)$ and apply its part $f_i(t)$.
- However, once we have disagreements on the load state and the applied force (due to noise and delays/desynchronization), the reconstructed global information becomes completely corrupted and unreliable.



Residual MARL: Proposed Solution

- Train a residual policy that corrects the controller action
- The goal is to find a policy that can still maintain reasonable load tracking and coordinated carrier trajectories despite:
 - Noisy and asynchronous load state measurements
 - Unreliable estimates of global information



Residual MARL

- Action Space: $\delta f \in \mathbb{R}^{12}$
 - Keep wrench controller and optimizer. Run decentralized

$$f(t) = f_g(t) + f_\lambda(t) + \underline{\delta f(t)}$$

- Better learning when decomposed

$$f(t) = G(t)^\dagger \cdot (w_d(t) + \underline{\delta w(t)}) + N(t) \cdot (\lambda(t) + \underline{\delta \lambda(t)})$$

$$\delta w \in \mathbb{R}^6, \delta \lambda \in \mathbb{R}^4$$

where $G(t)$ is the Grasp matrix, † denotes its pseudo-inverse.

$N(t)$ is in the Null space of $G(t)$ and $\lambda(t)$ is a function that is designed to shape the internal forces that redistribute cable tensions without affecting the load wrench.

All these variables are part of the centralized controller that we don't touch



Residual MARL

- Observation Space (Original) $\in \mathbb{R}^{44}$:
 - Measured load state $\in \mathbb{R}^{18}$
 $[p_L(t), R_L(t), \dot{p}_L(t), \omega_L(t)]$
 - Own carrier position and velocity $\in \mathbb{R}^6$
 $[p_R(t), \dot{p}_R(t)]$
 - Base force components
 $[f_g(t), f_\lambda(t)]$
 - Fourier time embedding/features
 $\phi(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \sin(\omega_2 t), \cos(\omega_2 t), \dots, \sin(\omega_k t), \cos(\omega_k t)]$



Residual MARL

- Observation Space (Augmented) $\in \mathbb{R}^{98}$:
- Original Observation Space in addition to:
 - The last two measured load states $\in \mathbb{R}^{24}$
$$[p_L(t-1), R_L(t-1), \dot{p}_L(t-1), \omega_L(t-1)]$$
$$[p_L(t-2), R_L(t-2), \dot{p}_L(t-2), \omega_L(t-2)]$$
 - The last two own carrier position and velocity $\in \mathbb{R}^{12}$
$$[p_R(t-1), \dot{p}_R(t-1)]$$
$$[p_R(t-2), \dot{p}_R(t-2)]$$
 - Reference load state at current time $\in \mathbb{R}^{18}$
$$[p_{Ld}(t), R_{Ld}(t), \dot{p}_{Ld}(t), \omega_{Ld}(t)]$$
- Did not lead to an improvement. History doesn't help hints that the policy is not trying to estimate "more correct" global information. This suggests that the policy is finding control actions that work despite loss of information.



Residual MARL: Implementation Details

- Multi-Agent Proximal Policy Optimization (MAPPO)
- Implemented MARL environment using PettingZoo
- Centralized Training Decentralized Execution (CTDE)
- Shared/Centralized critic with access to global state with a Parameter-Shared decentralized Actor
- Wrapped FMU generated by Simulink of the system model inside Gymnasium environment



Residual MARL

- Critic state $\in \mathbb{R}^{58}$
 - True load state $\in \mathbb{R}^{18}$
 $[p_L(t), R_L(t), \dot{p}_L(t), \omega_L(t)]$
 - All carriers' positions $\in \mathbb{R}^{12}$
 $[p_{R1}(t), p_{R2}(t), p_{R3}(t), p_{R4}(t)]$
 - All carriers' velocities $\in \mathbb{R}^{12}$
 $[\dot{p}_{R1}(t), \dot{p}_{R2}(t), \dot{p}_{R3}(t), \dot{p}_{R4}(t)]$
 - Expert-generated carrier position $\in \mathbb{R}^{12}$
 $[p_{Re1}(t), p_{Re2}(t), p_{Re3}(t), p_{Re4}(t)]$
 - Delay of each carrier load measurement $\in \mathbb{R}^4$
 $[d_1, d_2, d_3, d_4]$

Delays are added to critic state since they can make the state harder or easier, thus changing the expected reward, giving it to the critic helps making the advantage function more accurate.



Residual MARL

- Reward function
 - Design Methodology: Start simple and add terms by observing failure modes
 - Per drone reward is:

$$r_i = -\left(w_{manifold}d_i^2 + w_{stall}stall_i + w_{load}\|e_L\|^2 + w_{overspeed}overspeed_i^2 + w_{overshoot}overshoot_i\right)$$

- Team reward:

$$R_{team} = \sum_{i=1}^N r_i$$



Residual MARL

- Reward function terms breakdown:

$$r_i = - \left(w_{manifold} d_i^2 + w_{stall} stall_i + w_{load} ||e_L||^2 + w_{overspeed} overspeed_i^2 + w_{overshoot} overshoot_i \right)$$

Term	Expression	Role
manifold (d_i^2)	$ p_{expert,i}(t) - p_i(t) ^2$	Stay on the expert loiter loop (generated from ground truth states). The goal is not to follow it exactly, but to learn the underlying orbiting behavior in general.
stall_i	Graced hinge below cruise floor: $d^2 + 1.0 \cdot d$, where $d = \max(0, (\epsilon + \text{margin}) - v_i)$	Fixed-wing stall floor (prevents airspeed drop)
load tracking $e_L ^2$	$ e_L ^2$	Keep the load on reference
overspeed_i²	$\min(\text{over}^2 + 2 \cdot \text{over}, 75)$ where $\text{over} = \max(0.0, v_i - \text{overspeed_v})$	Prevent carrier from speeding up unnecessarily. Blowup guardrail.
overshoot_i	$\min(\text{over}_{\lambda,i}^2 + \text{over}_{\lambda,i}, 9)$ where $\text{over}_{\lambda,i} = \max(0, \text{sat}_{\lambda,i} - 1)$ and $\text{sat}_{\lambda,i} = \delta\lambda_i _2 / (\kappa_\lambda \cdot \lambda_{i,base} _2)$ κ_λ : the allowable fraction of the base controller magnitude	Prevents the policy from taking actions above the allowed fraction of the base controller magnitude (dead gradient, no learning). The cap prevents the policy from taking overly aggressive actions.

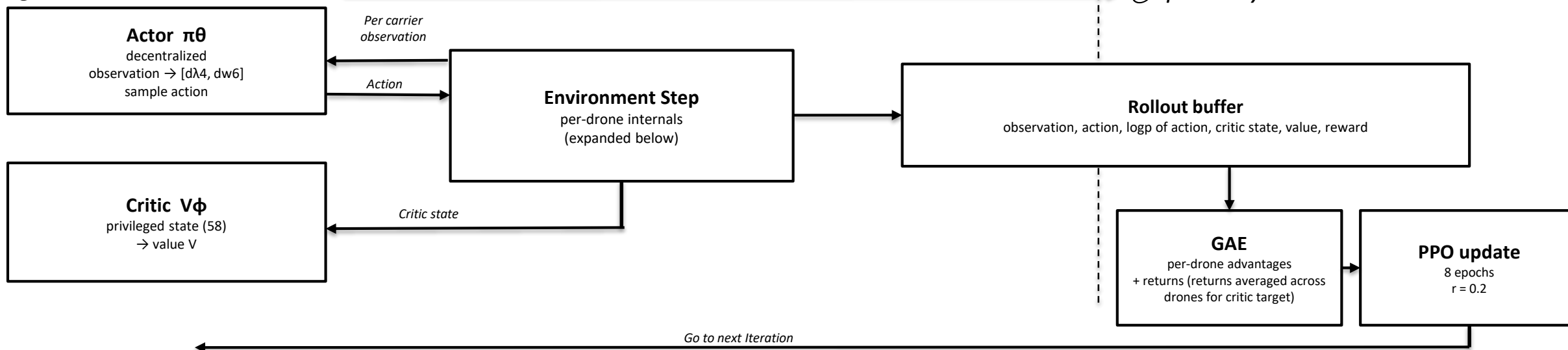
Truncation Condition: A one-shot penalty of -100 is triggered on a guard-truncated step if speed exceeds $v_{blowup} = 100$.



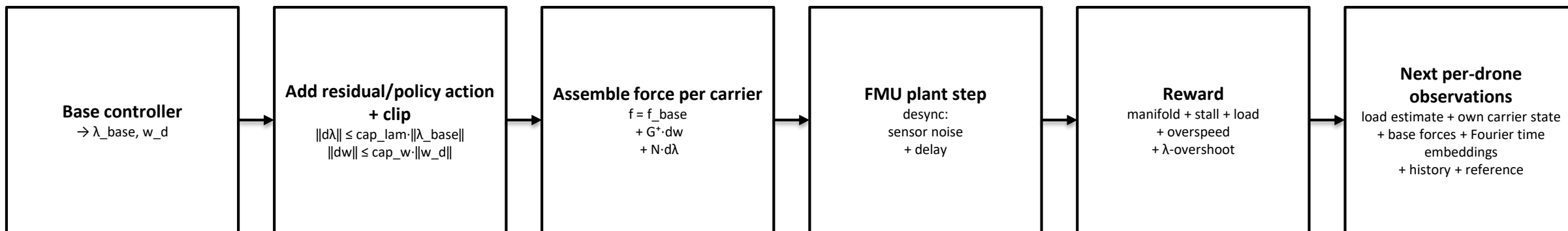
Training Loop + Environment Step

① Collect experience (multiple episodes)

② Update Policy



Environment step

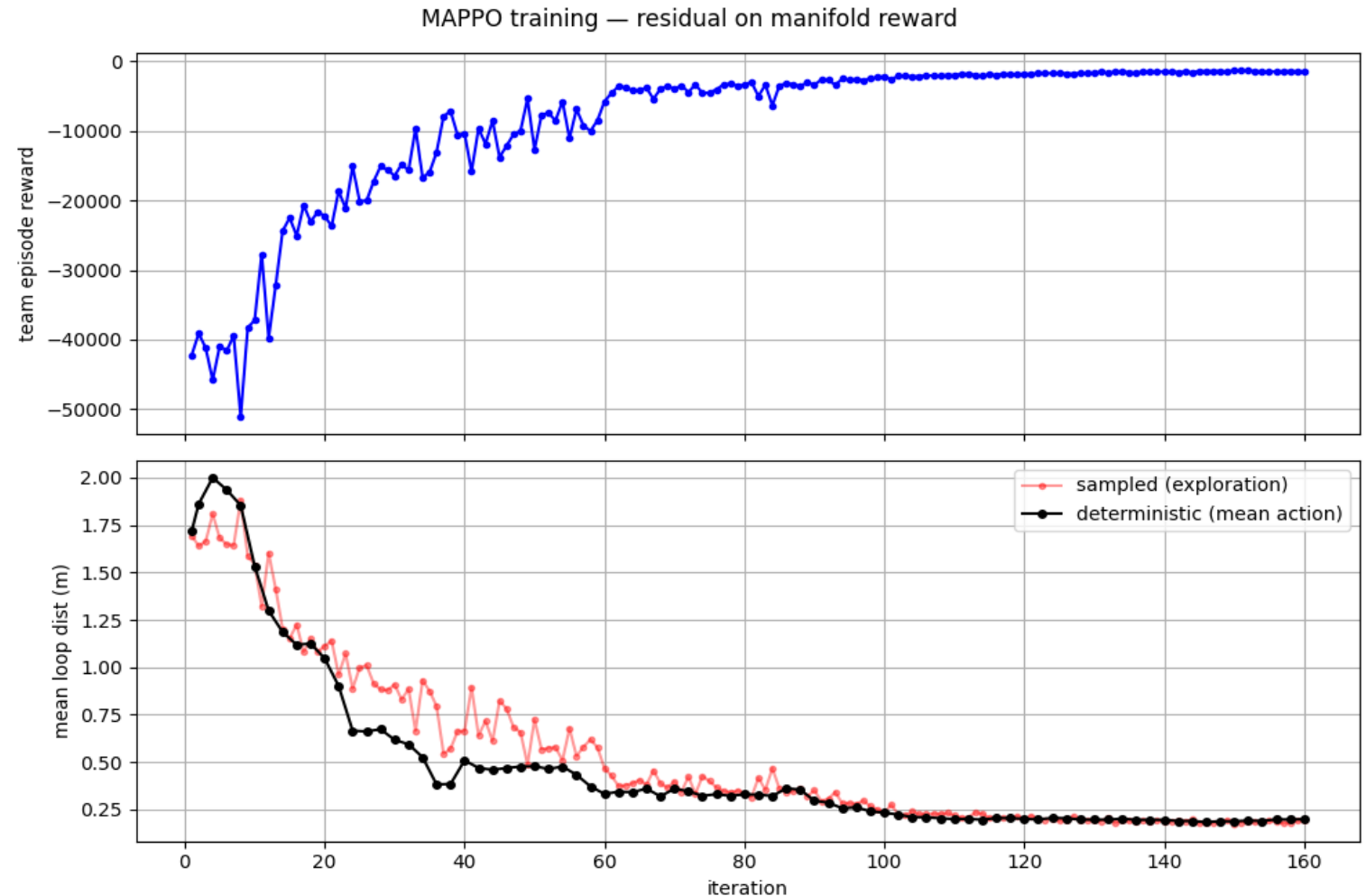


Do these three steps one time per each carrier then apply full force vector to the plant



Sample Results

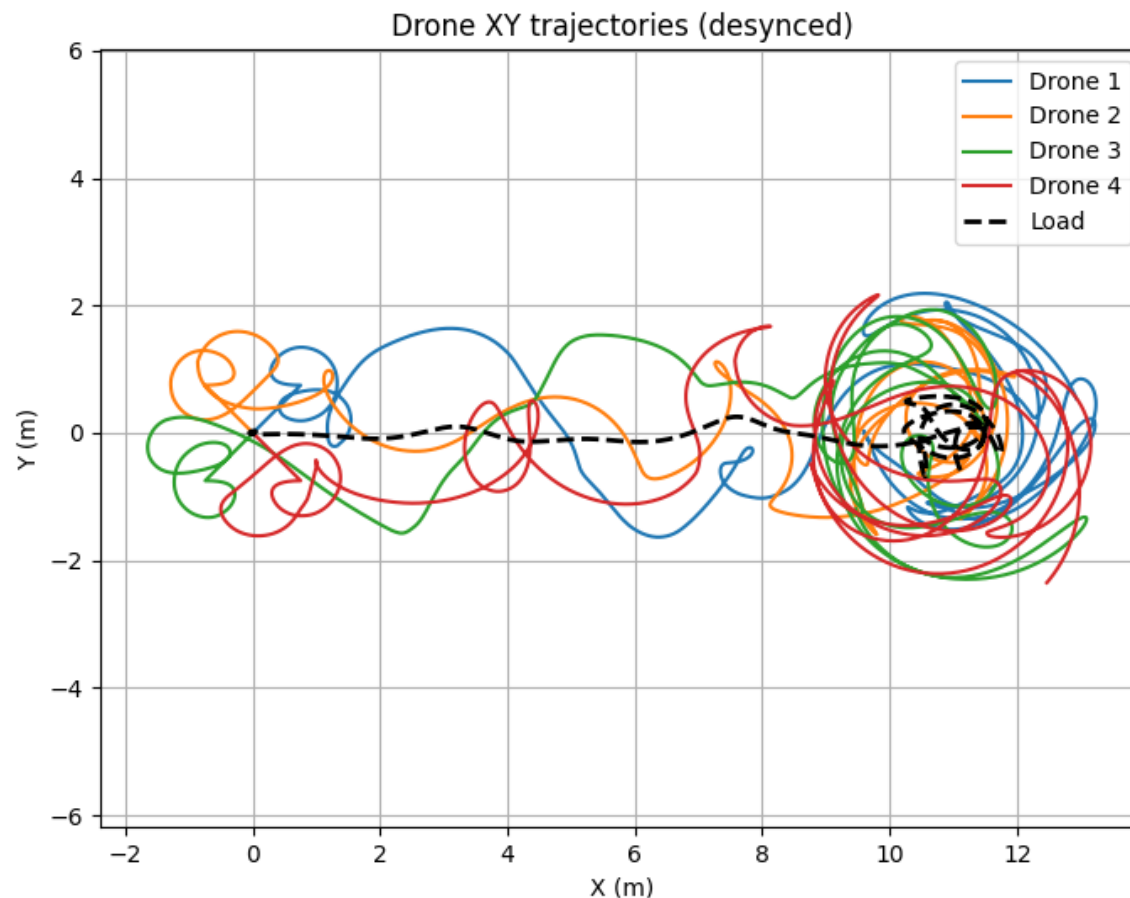
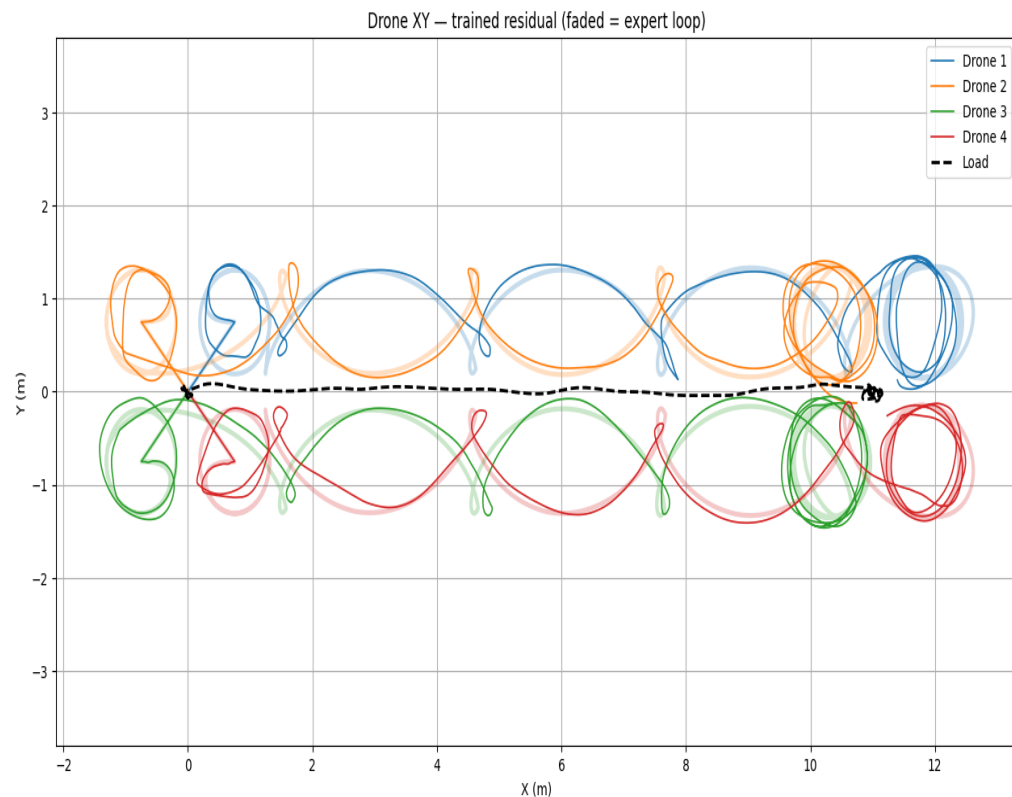
- Mean loop distance refers to the average distance between the carrier position and the expert noise free position (Term 1 in the reward).
- It is the most affected/degraded metric and is usually a good indicator on the overall health of the system

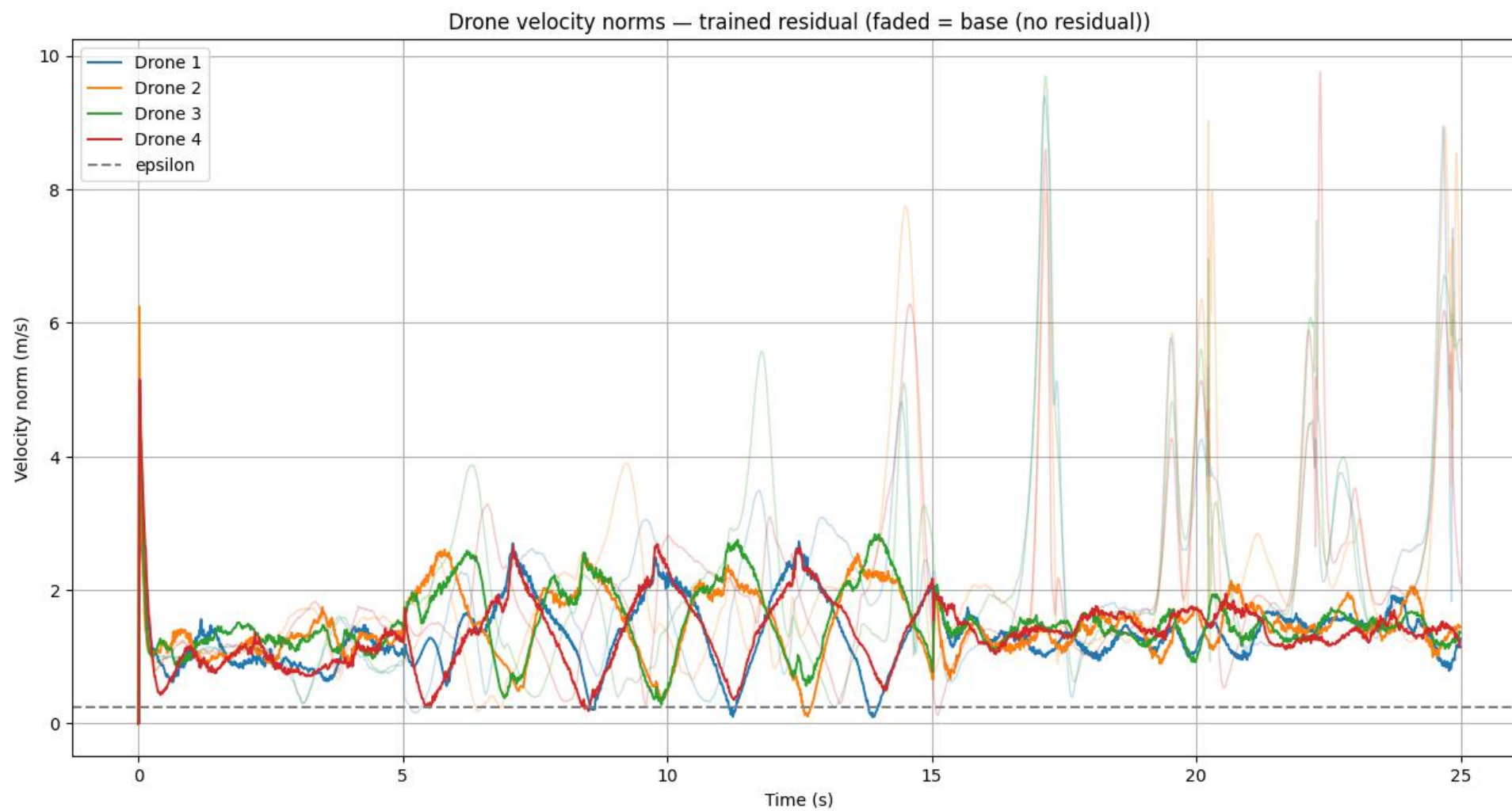




Sample Results: Trajectory 1: From training data

- With learned policy residual(left) vs. Base (no residual, right)

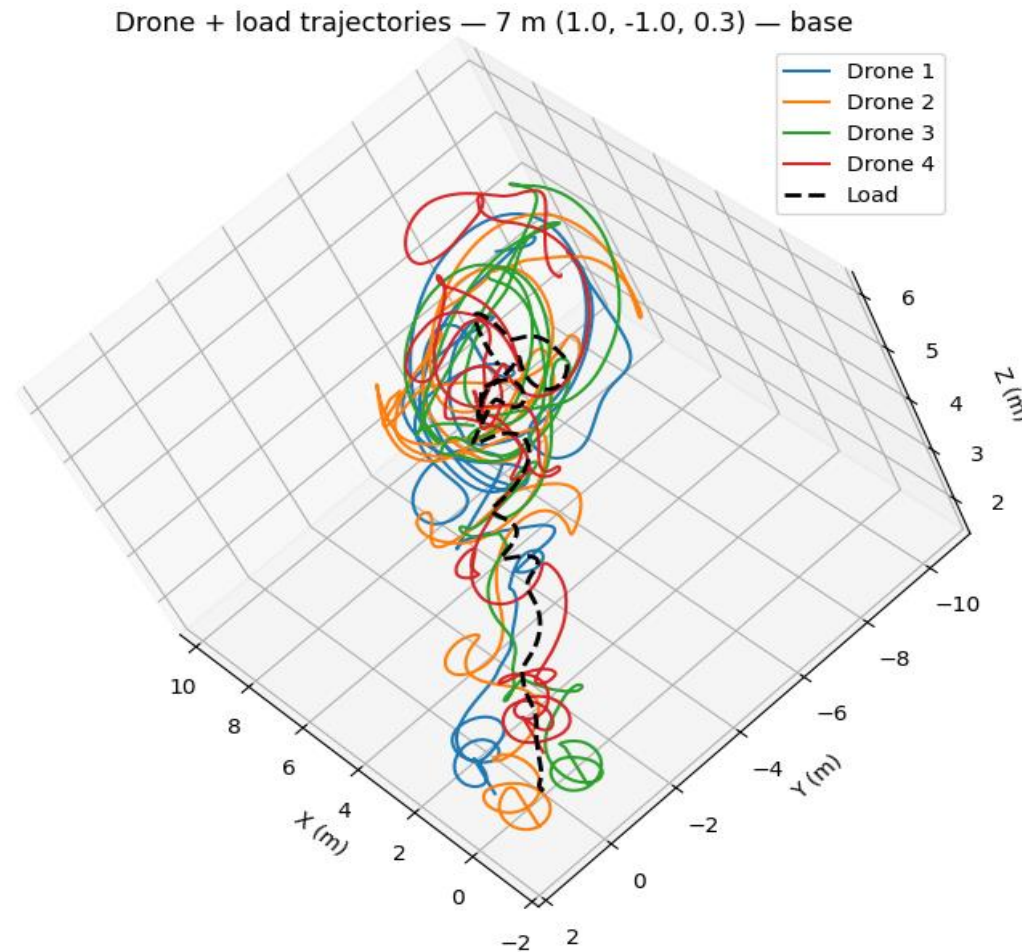






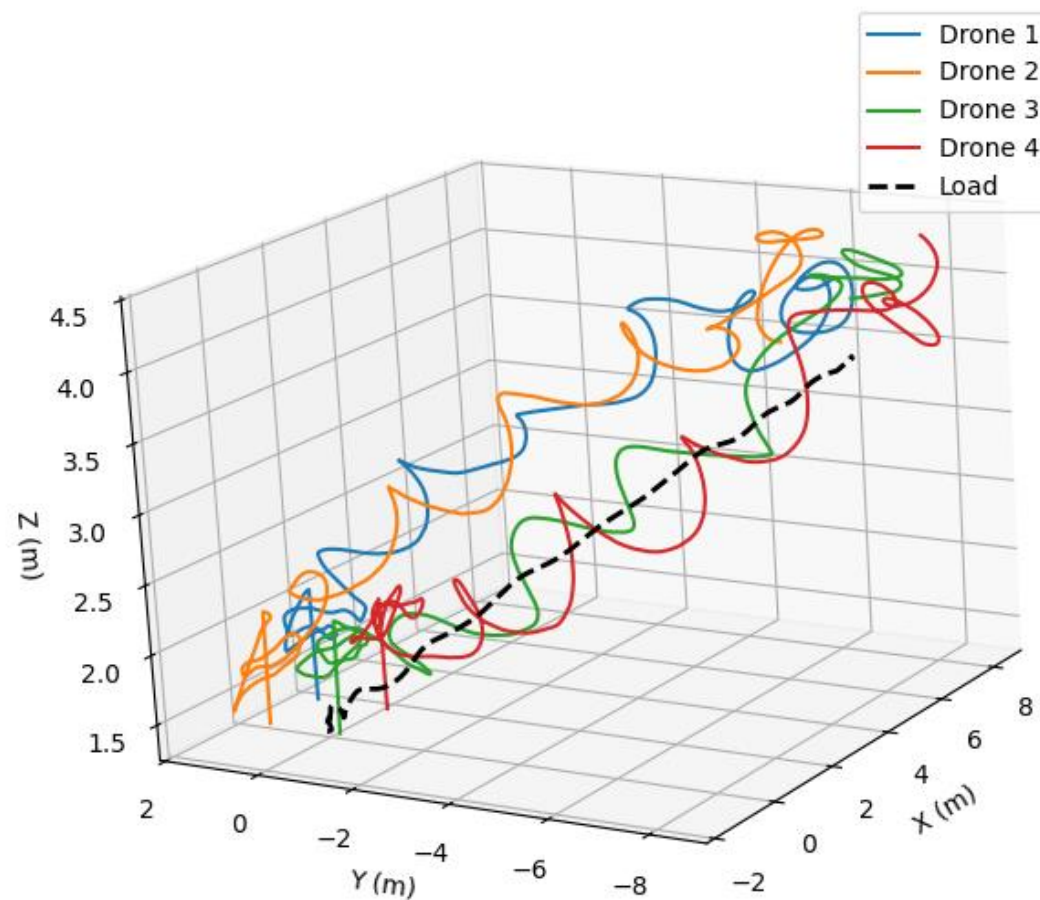
Trajectory 2: Unseen during training

- Magnitude: 7 meters
- Episode length: 27s
- Direction:
[1.0, -1.0, 0.3]



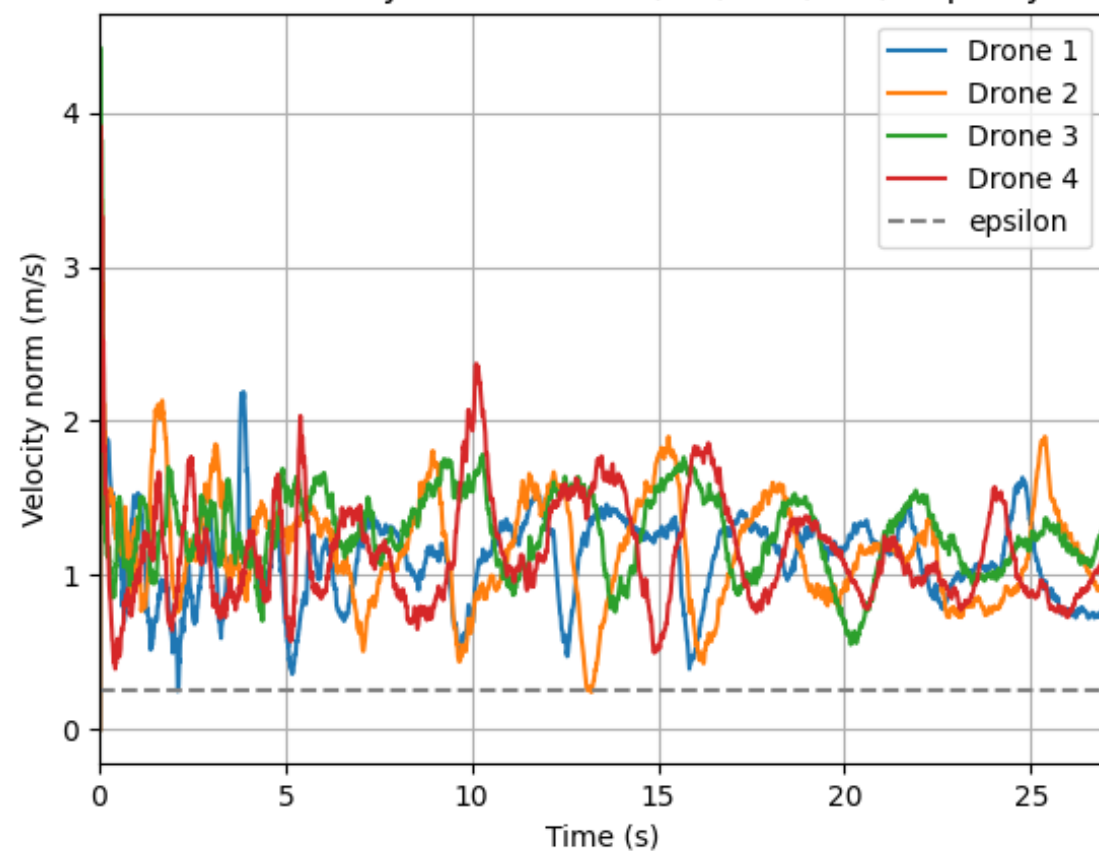


Drone + load trajectories — 7 m (1.0, -1.0, 0.3) — policy

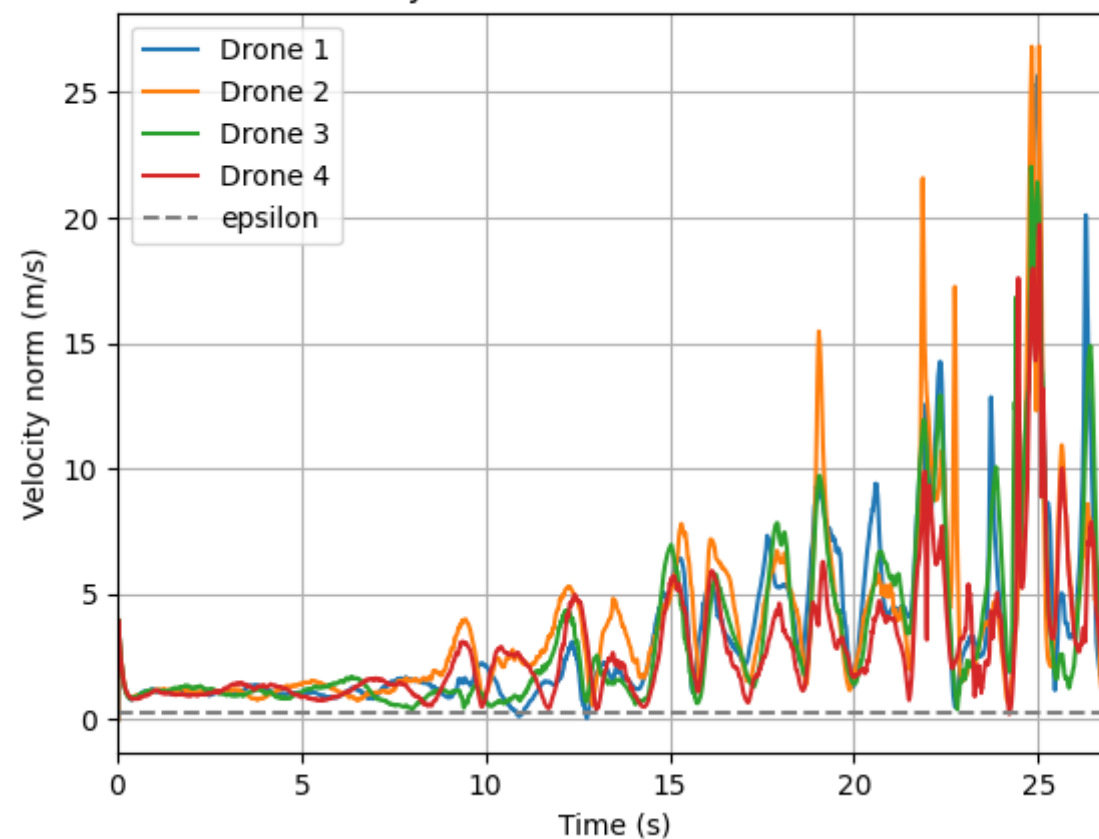




Drone velocity norms — 7 m (1.0, -1.0, 0.3) — policy

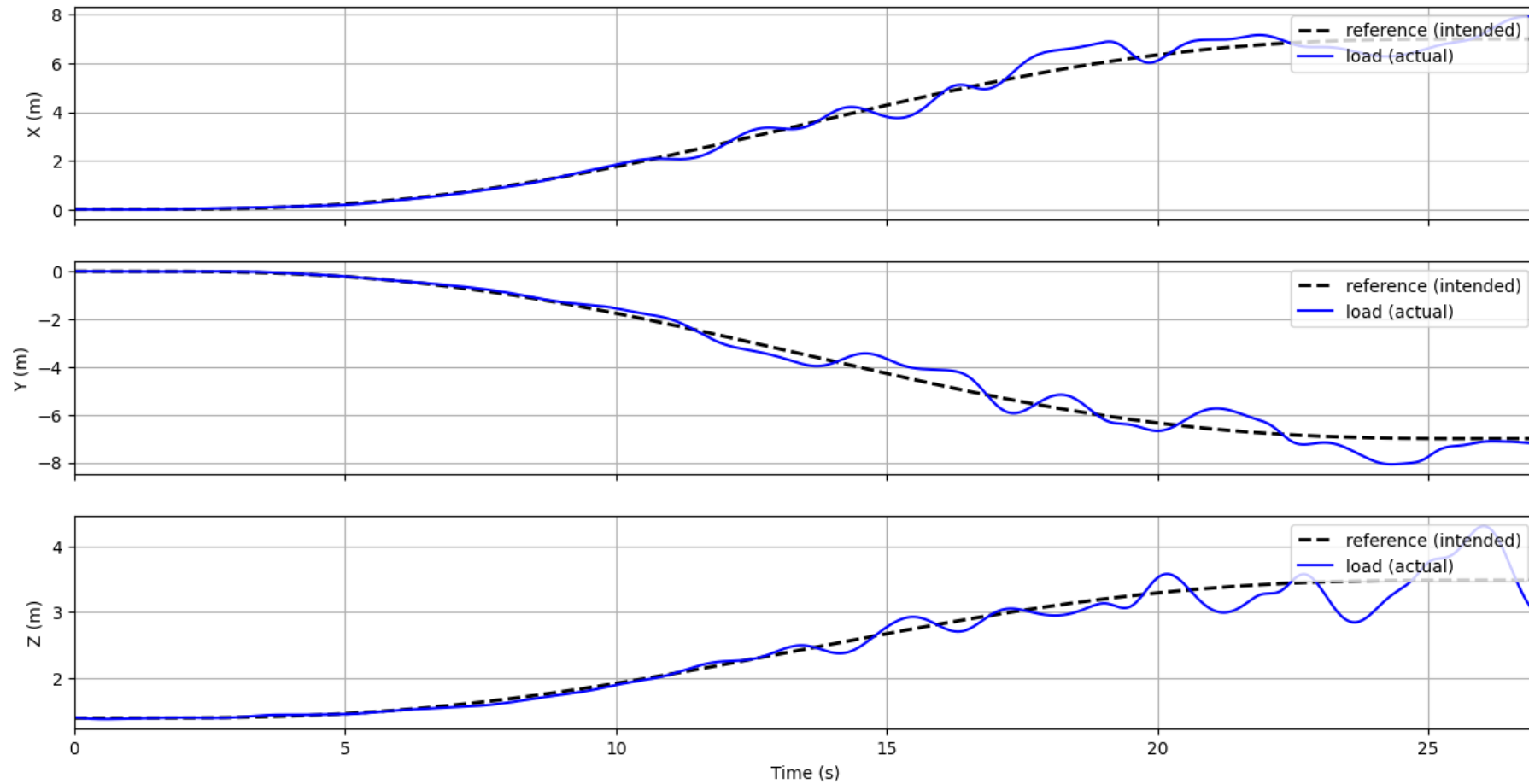


Drone velocity norms — 7 m (1.0, -1.0, 0.3) — base



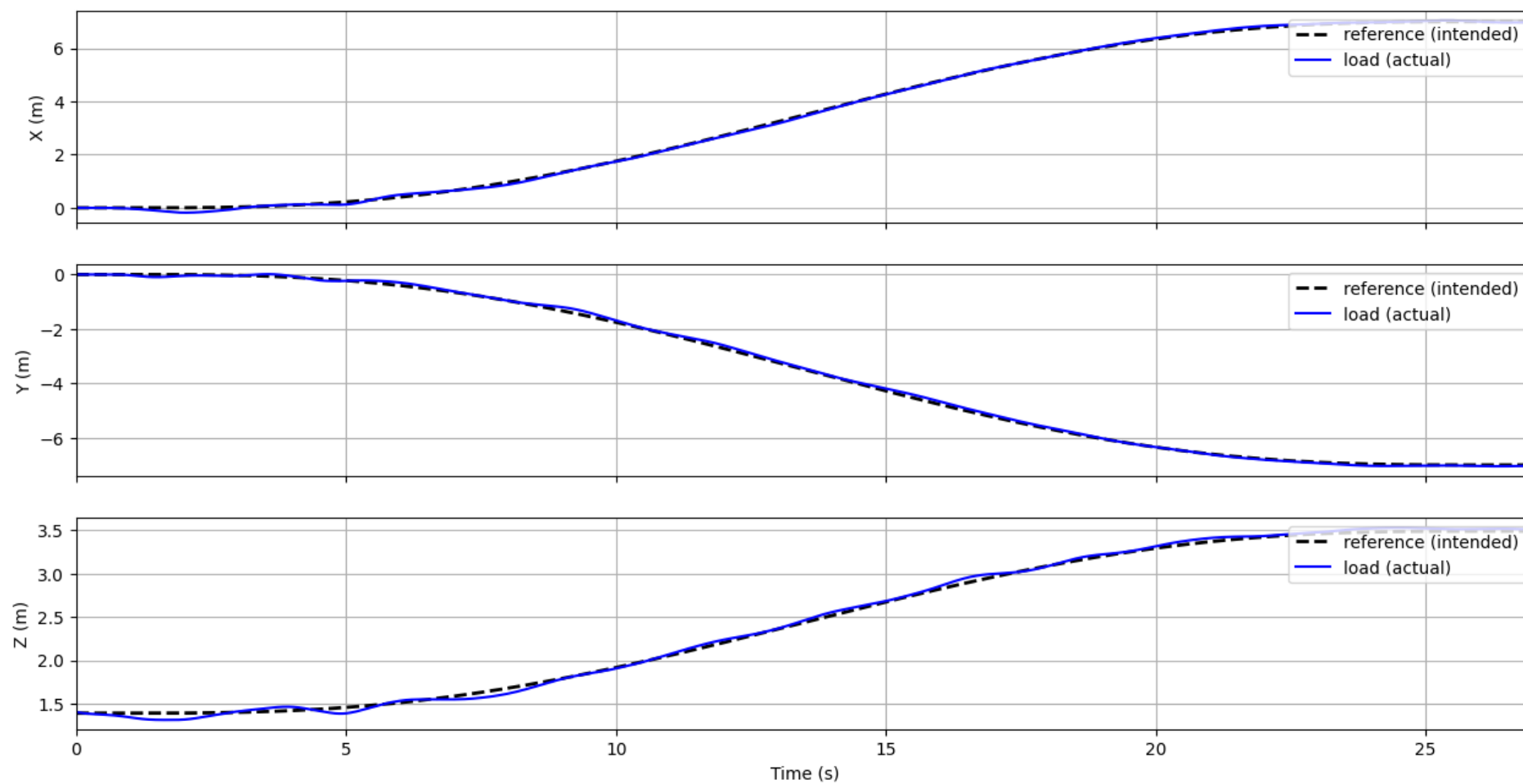


Load position — 7 m (1.0, -1.0, 0.3) — base





Load position — 7 m (1.0, -1.0, 0.3) — policy





Load and Expert* Tracking Performance: Trajectory 2

Metric	Policy	Base (No residual)
Expert loop MSE (m ²)	~0.1740	~3.0715
Load Track MSE (m ²)	~0.0083	~0.3058

*Expert tracking is not supposed to reach an MSE of zero, but it is just used as qualitative proxy for smooth orbiting behavior



Residual MARL: Status and Challenges remaining

- The policy is showing acceptable generalization results across:
 - Different episode lengths
 - Different movement magnitudes
 - Generally better than the base controller alone
- The policy is struggling to generalize well across various movement directions
 - Some directions exhibit significantly worse performance than others, even though they are all represented roughly equally in the training data.



Residual MARL: Status and Challenges remaining

- Training the policy specifically on these directions yields improvements, but at the cost of the other directions.
- Training on a wide combination of trajectories makes the reward extremely high variance making learning practically impossible.
- I was able to distill a combination of two policies using Behavioral Cloning in one net of the same size. Therefore, the issue is likely that MAPPO is not able to find a good generalizing strategy, but it does exist (at least something better than what it finds currently exists).



Residual MARL: Code

- [Github](#)
- [MAPPO](#)
- [MARL Environment](#)
- [Trajectory test](#)